

Introduction

L'objectif de ces travaux projet est d'ajouter des fonctionnalités supplémentaires au compilateur RAT → TAM codé lors des séances de TP.

En particulier, nous ajoutons le support de :

- Pointeurs
- Procédures (fonctions sans retour de valeur)
- Passage de variables par référence
- Enumérations

Nous détaillerons par la suite la répartition des rôles de chacun, les implémentations qui ont été faites, ainsi que quelques pistes d'amélioration.

Répartition des tâches

Emmanuel DUBOIS :

- Pointeurs : tests des passes de placement et de conversion de code
- Enumérations : code et tests

Aurélien VERBEKE :

- Pointeurs : code et tests des passes de gestion des identifiants et de typage
- Procédures : code et tests
- Références : code et tests

Pour chaque fonctionnalité, les tests ont été réalisés avant l'implémentation complète, afin de s'assurer que nous ne pourrions pas passer à côté d'erreurs (ou qu'en tout cas, nous ferions tout pour les minimiser).

Implémentation

Préliminaire : affectables

La mise en place d'affectables permet de mieux gérer l'affectation et la lecture de variables, en particulier lorsque nous introduirons la présence de pointeurs. Ainsi, un affectable peut être une variable à laquelle on affecte une variable, ou une variable dont on lit la valeur.

Modification de l'AST

On déplace l'unité `Ident` dans un nouveau type `affectable`, et la destination d'une affectation n'est plus simplement une variable mais un affectable.

Modification des passes

Les modifications consistent globalement en le déplacement du traitement des affectations ou lectures de variables dans une fonction dédiée à l'analyse des affectables.

Celle-ci devant avoir un comportement différent en lecture et en écriture, on crée un type `acces` qui peut être `AccesLecture` ou `AccesEcriture`, et on le passe en paramètre à la fonction d'analyse des affectables.

Aussi, lors de la passe de **gestion des identifiants**, une variable lue peut être une variable "classique" (affectable) ou une constante. Ainsi, on crée un type `schrodinger` qui peut être `SchrodingerEntier` ou `SchrodingerAffectable` (on ne sait pas trop tant qu'on n'y a pas regardé...). La fonction d'analyse des expressions se charge ensuite de séparer en `AstTds.Entier` ou `AstTds.Affectable` selon le cas.

Pointeurs

Un pointeur permet de gérer une variable par son adresse en mémoire. Ainsi, on peut modifier une variable extérieure à une fonction à l'intérieur de cette dernière.

Jugements de typage

On définit tout d'abord le type de pointeur `t*` qui pointe sur un type `t`, ainsi que les règles de typage associées :

$$\begin{aligned} \sigma \vdash \text{null} &: \text{Undefined}^* \text{ (Pointeur nul)} \\ \frac{\sigma \vdash \text{TYPE} : t}{\sigma \vdash (\text{new TYPE}) : t^*} &\text{ (Allocation mémoire)} \\ \frac{\sigma \vdash x : t^*}{\sigma \vdash (*x) : t} &\text{ (Déréférencement)} \\ \frac{\sigma \vdash x : t}{\sigma \vdash (\&x) : t^*} &\text{ (Adresse de variable)} \end{aligned}$$

Modification de l'AST et des types

Tout d'abord, on ajoute un type `Pointeur` qui permet de pointer n'importe quel type de base, ainsi qu'un pointeur lui-même.

Dans l'AST, on ajoute les expressions `New`, `Adresse` et `Null`, et le nouvel affectable `Deref` pour gérer la création de pointeurs, la récupération de l'adresse d'une variable, la définition d'un pointeur nul et la lecture ou la modification de la valeur pointée.

Modifications des passes

Pas de modification majeure, on ajoute les nouveaux cas dans les fonctions d'analyse des expressions et des affectables.

Passage de **conversion en code TAM** : la fonction d'analyse des affectables doit transmettre le type de l'affectable concerné pour que dans le cas d'un déréférencement, on puisse faire un `loadi` de la bonne taille après avoir récupéré l'adresse pointée.

Procédures

Les procédures sont des fonctions qui ne retournent pas de valeur. Elles permettent d'organiser le code en sous-parties réutilisables en ayant souvent des effets de bord.

Jugements de typage

$$\sigma, \text{void} \vdash \text{return} : \text{void}, []$$

L'instruction `return` dans une procédure, de même que dans une fonction, n'est pas typée.

Modification de l'AST et des types

On ajoute le type `Void` pour représenter l'absence de type de retour.

On crée un `AppelProcédure` différent de `AppelFonction` pour ne pas confondre entre l'expression et l'instruction, et on crée une fonction d'analyse des appels annexe qui évitera les duplications de code entre les deux.

Pour le `Retour` : on choisit de créer une expression `RetourVoid` pour les procédures. Cela évite les `if` à rallonge dans les analyses de `Retour`.

Modifications des passes

Une procédure étant une fonction sans type de retour, les modifications consistent principalement à gérer le type `Void` dans les différentes passes.

Ainsi, on s'assure que :

- une procédure ne peut pas retourner de valeur
- une procédure ne peut pas être utilisée dans une expression
- le type `void` est uniquement autorisé comme type de retour d'une procédure (pas dans les paramètres ou dans une déclaration de variable)

Choix non retenus

On ne crée pas un type `InfoProcedure` équivalent de `InfoFun`. Cela impliquerait une duplication des analyses de fonction dans toutes les passes. On fait tout à partir du type de retour qui sera donc `Void`.

Références

Toute la difficulté réside dans le fait que l'on peut transmettre des variables à des emplacements mémoire différents pour une même fonction qui doit avoir des emplacements mémoires intrinsèques fixes. On choisit donc de traiter les références comme des pointeurs.

Jugements de typage

$$\frac{\sigma \vdash x : t}{\sigma \vdash \text{ref } x : t}$$

Une référence à une variable de type `t` est de type `t`.

Modification de l'AST et de la table des symboles

On ajoute l'expression `Ref` pour gérer la création de références.

On modifie également la liste des paramètres d'une fonction en ajoutant un booléen indiquant si un paramètre est passé par référence ou par valeur.

Enfin, on ajoute le booléen de présence de référence dans les informations des fonctions. Ceci permettra de savoir si on a bien transmis une référence lorsqu'une est attendue, ou inversement.

Modifications des passes

Passe de **gestion des identifiants** : on vérifie que les références sont bien passées sur des variables (hors constantes), et qu'elles ne sont pas des sous-expression d'une autre expression.

Passe de **typage** : on vérifie qu'une référence est bien passée à un paramètre qui attend une référence, ou inversement. Comme cette vérification est faite dans l'analyse de l'appel à une fonction ou une procédure, nous n'avons accès qu'à l'`InfoFun` de la fonction appelée. Ceci prouve donc la nécessité d'avoir le booléen de présence de référence dans les informations des fonctions.

Passe de **placement mémoire** : pas de modifications majeures. Il est juste à noter qu'une référence n'occupe qu'un espace mémoire, puisqu'on la traite comme un pointeur.

Passe de **conversion en code TAM** : à tout moment, il faut savoir si une variable est une référence ou non, pour savoir si on doit la traiter comme une simple variable ou comme un pointeur. Ainsi, on transmet à chaque fonction la liste des `info_ast` des variables passées en paramètres. Dans le cas d'une fonction, on utilise la liste présente dans l'`InfoFun`, et dans le cas du bloc principal, on transmet une liste vide.

Choix non retenus

On aurait pu créer un type référence, et les traiter comme des pointeurs. Cependant, il aurait fallu convertir les instructions de références en pointeurs à un moment donné, et cela aurait complexifié la passe de typage. Aussi, introduire un nouveau type uniquement interne aurait pu prêter à confusion.

Enumérations

Les types énumérés sont des types personnalisés définis avant le bloc de définition des fonctions. Les variables de ce type peuvent prendre plusieurs valeurs, qui doivent être uniques parmi tous les types énumérés. De même, deux types énumérés ne peuvent pas avoir le même nom.

Modification du lexer et du parser

Afin de pouvoir traiter l'ajout des types énumérés, il est nécessaire de rajouter l'expression régulière associée au token `tid`, qui définit un identifiant où la première lettre est nécessairement en majuscules. On rajoute également le token `ENUM`, qui permet ensuite au parser de détecter une définition de type énuméré.

Jugements de typage

Nouveau type : `Enum n` avec `n` un descripteur nommé de l'énumération.

$\sigma \vdash \text{enum } n \{DP\} :: \text{void}, [(n, \text{Enum } n) ; (n_1, \text{Enum } n) ; \dots ; (n_n, \text{Enum } n)]$ (Définition d'un type énuméré, $DP = n_1, \dots, n_n$)

$$\frac{\sigma \vdash x : \text{Enum } n_1 \quad \sigma \vdash y : \text{Enum } n_2}{\sigma \vdash x = y : \text{Bool}} \text{ (Egalité de valeurs de types énumérés)}$$

Modification de l'AST, des types et de la table des symboles

On ajoute dans le type info deux nouvelles infos :

- `InfoValEnum` afin de stocker le nom du type énuméré de la variable, son nom ainsi que l'indice auquel sa valeur apparaît dans la liste des valeurs du type énuméré
- `InfoEnum` qui stocke le nom du type énuméré ainsi que la liste des valeurs

Il est également nécessaire de rajouter le type `Enum` afin de permettre les comparaisons entre des variables du même type énuméré.

Finalement, on ajoute l'expression `Enum` afin de gérer l'affectation et la déclaration de valeurs de type énuméré.

Modification des passes

Passe de **gestion des identifiants** : on vérifie que les expressions sous forme de tid aient déjà bien été déclarées, car elles doivent déjà exister dans la TDS comme elles ont été enregistrées lors de la déclaration des types énumérés. On vérifie également que le token vérifié est bien une valeur de type énuméré et non un nom de type énuméré. On ajoute également une méthode pour inscrire dans la table des symboles principale le nom de chaque type énuméré déclaré ainsi que toutes ses valeurs.

Passe de **typage** : on ajoute ici la surcharge de l'opérateur égal pour les types énumérés, en comparant les valeurs numériques stockées dans les `InfoValEnum`. Comme les déclarations de types énumérés ont déjà été traitées, il n'y a pas besoin de les traiter dans cette passe.

Passe de **placement mémoire** : pas de modifications majeures. Il faut noter qu'une valeur de type énuméré n'occupe qu'un espace mémoire, comme on la traite comme un entier en mémoire.

Passe de **conversion en code TAM** : lorsqu'on rencontre une expression désignant une valeur d'un type énuméré, il nous suffit de récupérer la valeur numérique dans l' `InfoValEnum` et de l'inscrire dans le registre via un `loadl_int`.

Conclusion

Ce projet nous a permis de mieux comprendre le fonctionnement d'un compilateur, en particulier l'implémentation de certains mécanismes essentiels comme les pointeurs, références et énumérations.

Nous avons certaines fois exploré certaines pistes qui se sont avérées non concluantes, mais qui nous ont toutefois permis de mieux cerner les enjeux de chaque fonctionnalité.

Pistes d'amélioration

- ☐ Affichage des adresses mémoire
- ☐ Ne pas autoriser le type de retour `void*` ou `void**` ou ...